fa4d61d6-e774-476e-ab42-f3d8c139c88b

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'microsoft/typescript-go' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/typescript-go/typescript-go (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-06-13T01:14:52.922+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Fix declaration emit panic for CommonJS named exports
</pr_title>

<pr_description>
CommonJS declaration emit could panic while reporting symbol accessibility diagnostics for inferred named export types that reference inaccessible symbols.

The named non-default CJS export path needed explicit diagnostic-context handling:

- **Named non-default exports**: `transformCommonJSExport` now sets `getSymbolAccessibilityDiagnostic` for named non-default

exports such as `exports.foo = ...`, matching the default-export and non-identifier-name branches.

Tests added:

- `cjsModuleExportsNamedDeclarationEmit.ts` covers named non-default CJS export declaration emit.
</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_reviews>

<comment_thread_resolved>
<file>internal/transformers/declarations/transform.go:1100-1106</file>
<original_commit>c396f84</original_commit>

</comment_thread_resolved>

</pr_reviews>

<pr_comments>

<comment_new>
<comment_id>4425665109</comment_id>
<author>@jakebailey</author>
@copilot+claude-opus-4.7 If you undo your fixes, your test doesn't actually crash.
</comment_new>

<comment_new>
<comment_id>4685512856</comment_id>
<author>@jakebailey</author>
@copilot+gpt-5.5 merge main and try again. try as hard as you can to find a crash. do not guess at a fix.
</comment_new>

n emit variants for the original `Diagnostic emitted without context` panic and did not find it.
- I did find a related real crash for `module.exports = { micromatch }`: `ensureType` replaced the export-assignment diagnostic context with a binary-expression context, producing TS4032 with too few formatting args and panicking with `Invalid formatting placeholder`.

The committed fix is narrow, it preserves the export-assignment diagnostic context while serializing the synthetic `_default` type. No speculative node-reuse changes.
</comment_old>

<comment_new>
<comment_id>4694207932</comment_id>
<author>@jakebailey</author>
@copilot+gpt-5.5 I have no idea what you're talking about. This PR is supposed to fix #3378
</comment_new>

<comment_old>
<author>@copilot</author>

> @copilot+gpt-5.5 I have no idea what you're talking about. This PR is supposed to fix #3378

Refocused the PR on #3378 in e791522dd: removed the unrelated object-form export-assignment change/test and left the named CJS export diagnostic-context fix with its regression test. I also ran `npx hereby build`, `npx hereby test`, `npx hereby lint`, and `npx hereby format` successfully.
</comment_old>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>internal/transformers/declarations/transform.go:277-299</file>
<original_commit>eec8c70</original_commit>

<comment_new>
<comment_id>3406617473</comment_id>
<author>@jakebailey</author>
This is https://github.com/microsoft/TypeScript/blob/c3bd12d888b86f676718b16e64d7d2abcb423514/src/compiler/

```
transformers/declarations.ts#L447-L457
</comment_new>
```

</comment_thread>

</pr_reviews>

<pr_comments>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
Pull request overview

Fixes a declaration emit panic when symbol accessibility diagnostics are reported for certain CommonJS export patterns in JavaScript/CommonJS modules, by ensuring the declaration transformer always has a valid diagnostic context in JS files.

Changes:
- Install a JS-file-scoped `getSymbolAccessibilityDiagnostic` fallback in `DeclarationTransformer.visitSourceFile` to avoid panics when declaration emit reports private-name accessibility issues without a per-node diagnostic context.
- Add a new compiler regression test covering `Object.defineProperty(exports, "...")` exporting a value whose inferred type references a private name from another module.
- Add reference baselines (`.types`, `.symbols`, `.errors.txt`) for the new test.

Reviewed changes

Copilot reviewed 5 out of 5 changed files in this pull request and generated 1 comment.

</comment_old>

</pr_comments>

<pr_reviews>

<comment_thread>

<file>internal/transformers/declarations/transform.go:277-285</file>
<original_commit>eec8c70</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
The PR description says the fix is in `transformCommonJSExport` for named non-default exports (e.g. `exports.foo = ...`) and that it adds `cjsModuleExportsNamedDeclarationEmit.ts`, but this diff instead installs a JS-file-wide `getSymbolAccessibilityDiagnostic` override in `visitSourceFile` and adds `cjsObjectDefinePropertyPrivateModuleDeclarationEmit.ts`. Please either update the PR description (and test name reference) to match this implementation, or include the described `transformCommonJSExport` change/test so reviewers can validate the intended scenario.
</comment_old>

<comment_new>
<comment_id>3406635684</comment_id>
<author>@jakebailey</author>
I reset this PR from scratch and did my own copiloting, so it's not up to date anymore unfortunately
</comment_new>

<comment_new>
<comment_id>3406635684</comment_id>
<author>@weswigham</author>
There is no way this is the right fix (and the review here is at least observing how it's wrong) - an assignment at the top level? No. It's supposed to be set inline before you invoke a resolver function - it's left as `nil` by default specifically so unhandled cases crash (ideally in test). (So we don't have a generic "there was an error somewhere" error.) I imagine this assignment should be *much* simpler and be inside one of the new expando transform functions instead...
</comment_new>

<comment_new>
<comment_id>3406635684</comment_id>
<author>@weswigham</author>
@copilot remove the global set and add the missing set before the resolver calls that need it set. It's intentionally set lazily so context can be provided for the

```
diagnostic - see other resolver calls for context.
</comment_new>
```

</comment_thread>

</pr_reviews>

</comments>

The last **2** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement><rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. Treat <comment_old> comments as context only and do not make changes solely to satisfy them.
* Only act on <comment_new> comments that explicitly mention you (@copilot). To understand the context of comments that explicitly mention you, you may use other comments that do not explicitly mention you. Do not act on comments that don't explicitly mention you or reply to them, even if they are marked as <comment_new>.
* If the PR author has pushed back on or rejected a suggestion in a thread, do not act on that thread.
* For each comment you decide to act on, determine:
    - If it is a request for changes, a question on the code, or a suggestion.
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

<system_reminder>

</system_reminder>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.

<prev_assistant>

Done — I committed and pushed the pending changes.

Latest commit: `d67f7a4d`
- Removed the global JS diagnostic-context assignment.
- Added lazy, localized diagnostic-context setup before the relevant CommonJS named-export resolver call, and restored the previous context after the call.
- Preserved `result.ErrorNode`-based contextual diagnostics when available.

I also:
- Ran final required checks successfully:
  - `npx hereby build`
  - `npx hereby test`
  - `npx hereby lint`
  - `npx hereby format`
- Ran `parallel_validation` (no CodeQL alerts; one non-blocking style suggestion about helper extraction).
- Replied to the actionable PR comment (`commentId: 3406635684`) with the commit hash.

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'microsoft/typescript-go' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/typescript-go/typescript-go (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-06-13T01:49:42.070+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Fix declaration emit panic for CommonJS named exports
</pr_title>

<pr_description>
CommonJS declaration emit could panic while reporting symbol accessibility diagnostics for inferred named export types that reference inaccessible symbols.

The fix keeps diagnostic context lazy and scoped to the exact resolver call that needs it in the named non-default CJS export path:

- **Named non-default exports**: `transformCommonJSExport` now sets `getSymbolAccessibilityDiagnostic` immediately before the relevant `ensureType(...)` call and restores the previous callback afterward.
- When available, it uses `result.ErrorNode` to preserve node-specific diagnostic context; otherwise it falls back to the JS declaration-emit private-name diagnostic for this path.
- The previous global JS-file diagnostic-context override was removed.

Tests/baselines:

- `cjsObjectDefinePropertyPrivateModuleDeclarationEmit.ts` covers the affected CommonJS named export path (`Object.defineProperty(exports, "api", ...)`).
- Updated the corresponding `.errors.txt` baseline to match the corrected diagnostic span.
</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_reviews>

<comment_thread_resolved>
<file>internal/transformers/declarations/transform.go:1100-1106</file>
<original_commit>c396f84</original_commit>

<comment_old>

<author>@jakebailey</author>
This is technically correct (I had the same change locally), but not complete, because while adding the error stops the crash, there shouldn't be an error in the first place, and stopping the error requires further changes related to JS emit and how we choose to reuse nodes / resolve things even though they're in JSDoc. I have that change locally, but it still felt hacky (hence not having sent anything yet).
</comment_old>

<comment_old>
<author>@jakebailey</author>
@copilot+claude-opus-4.7 please consider my comment above; think about how to make the error not happen at all with the repro provided in the issue, as that is the root issue in addition to this code
</comment_old>

<comment_old>
<author>@copilot</author>
Added a deeper fix in `finalizeBoundary` (22069e7) that addresses the root cause. When `finalizeBoundary` replays tracked symbols through the outer tracker, it now pre-checks each symbol's accessibility. If any symbol is not accessible from the enclosing declaration, the boundary is considered failed, causing `tryReuseExistingNodeHelper` to return nil and the caller to fall back to structural type serialization via `typeToTypeNode`. This prevents the accessibility error from being produced in the first place, since the type gets serialized without trying to reference the inaccessible symbol by name.

Both fixes are now in place:
1. The diagnostic handler (crash prevention, in case the pre-check doesn't catch all cases)
2. The `finalizeBoundary` pre-check (prevents the error from happening at all)
</comment_old>

</comment_thread_resolved>

</pr_reviews>

<pr_comments>

<comment_new>
<comment_id>4425665109</comment_id>
<author>@jakebailey</author>

<comment_new>
@copilot+claude-opus-4.7 If you undo your fixes, your test doesn't actually crash.
</comment_new>

<comment_old>
<author>@copilot</author>

> @copilot+claude-opus-4.7 If you undo your fixes, your test doesn't actually crash.

You're right — confirmed by reverting the fixes locally, the test passes without crashing. I tried several variations (inner class with JSDoc reference, @typedef referencing private types, cross-file references) and none of them reproduced the panic in corsa, since the inner `trackExistingEntityName` accessibility check at `nodecopy.go:350` catches inaccessible symbols before they ever get recorded for replay.

I've dropped the speculative `finalizeBoundary` pre-check (0e1296b) since I couldn't validate it with a real repro, and per your hint the right deeper fix lives in JS emit / node reuse / JSDoc resolution rather than in the boundary replay. The simple structural fix in `transformCommonJSExport` remains. The test is now just a sanity check for the named non-default CJS export emit path rather than a regression test for the crash. If you can share a repro that actually crashes, I can use it to drive the deeper fix.
</comment_old>

<comment_new>
<comment_id>4685512856</comment_id>
<author>@jakebailey</author>
@copilot+gpt-5.5 merge main and try again. try as hard as you can to find a crash. do not guess at a fix.
</comment_new>

<comment_old>
<author>@copilot</author>

> @copilot+gpt-5.5 merge main and try again. try as hard as you can to find a crash. do not guess at a fix.

Merged main and tried again in e49371740.

Findings:
- The original #2522 repro no longer crashes on merged main even with my named-export handler removed, because JSDoc type-node reuse now emits `typeof import("micromatch") | undefined`.

- I searched 1,080 micromatch-derived CJS declaration emit variants for the original `Diagnostic emitted without context` panic and did not find it.
- I did find a related real crash for `module.exports = { micromatch }`: `ensureType` replaced the export-assignment diagnostic context with a binary-expression context, producing TS4032 with too few formatting args and panicking with `Invalid formatting placeholder`.

The committed fix is narrow, it preserves the export-assignment diagnostic context while serializing the synthetic `_default` type. No speculative node-reuse changes.
</comment_old>

<comment_new>
<comment_id>4694207932</comment_id>
<author>@jakebailey</author>
@copilot+gpt-5.5 I have no idea what you're talking about. This PR is supposed to fix #3378
</comment_new>

<comment_old>
<author>@copilot</author>

> @copilot+gpt-5.5 I have no idea what you're talking about. This PR is supposed to fix #3378

Refocused the PR on #3378 in e791522dd: removed the unrelated object-form export-assignment change/test and left the named CJS export diagnostic-context fix with its regression test. I also ran `npx hereby build`, `npx hereby test`, `npx hereby lint`, and `npx hereby format` successfully.
</comment_old>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>internal/transformers/declarations/transform.go:277-299</file>
<original_commit>eec8c70</original_commit>

<comment_old>
<author>@jakebailey</author>
This is https://github.com/microsoft/TypeScript/blob/c3bd12d888b86f676718b16e64d7d2abcb423514/src/compiler/transformers/declarations.ts#L447-L457

</comment_old>

</comment_thread>

</pr_reviews>

<pr_comments>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
Pull request overview

Fixes a declaration emit panic when symbol accessibility diagnostics are reported for certain CommonJS export patterns in JavaScript/CommonJS modules, by ensuring the declaration transformer always has a valid diagnostic context in JS files.

Changes:
- Install a JS-file-scoped `getSymbolAccessibilityDiagnostic` fallback in `DeclarationTransformer.visitSourceFile` to avoid panics when declaration emit reports private-name accessibility issues without a per-node diagnostic context.
- Add a new compiler regression test covering `Object.defineProperty(exports, "...")` exporting a value whose inferred type references a private name from another module.
- Add reference baselines (`.types`, `.symbols`, `.errors.txt`) for the new test.

Reviewed changes

Copilot reviewed 5 out of 5 changed files in this pull request and generated 1 comment.

</comment_old>

</pr_comments>

<pr_reviews>

<comment_thread>

<file>internal/transformers/declarations/transform.go:277-285</file>
<original_commit>eec8c70</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
The PR description says the fix is in `transformCommonJSExport` for named non-default exports (e.g. `exports.foo = ...`) and that it adds `cjsModuleExportsNamedDeclarationEmit.ts`, but this diff instead installs a JS-file-wide `getSymbolAccessibilityDiagnostic` override in `visitSourceFile` and adds `cjsObjectDefinePropertyPrivateModuleDeclarationEmit.ts`. Please either update the PR description (and test name reference) to match this implementation, or include the described `transformCommonJSExport` change/test so reviewers can validate the intended scenario.
</comment_old>

<comment_old>
<author>@jakebailey</author>
I reset this PR from scratch and did my own copiloting, so it's not up to date anymore unfortunately
</comment_old>

<comment_old>
<author>@weswigham</author>
There is no way this is the right fix (and the review here is at least observing how it's wrong) - an assignment at the top level? No. It's supposed to be set inline before you invoke a resolver function - it's left as `nil` by default specifically so unhandled cases crash (ideally in test). (So we don't have a generic "there was an error somewhere" error.) I imagine this assignment should be *much* simpler and be inside one of the new expando transform functions instead...
</comment_old>

<comment_old>
<author>@weswigham</author>
@copilot remove the global set and add the missing set before the resolver calls that need it set. It's intentionally set lazily so context can be provided for the diagnostic - see other resolver calls for context.
</comment_old>

<comment_new>
<comment_id>3406635684</comment_id>
<author>@weswigham</author>

Like, it's funny, because the PR description is basically right - the implementation that was here was just way too broad to match that.

@copilot To be clear, the root cause is since `transformExpandoMember` and `transformExpandoHost` no longer go thru `transformDeclarationSubtree` after recent fixes, the diagnostic handler doesn't get set. The set logic from `transformDeclarationSubtree` (everything from `canProdiceDiagnostic` up to the `shouldEnterSuppressNewDiagnosticsContextContext` block, plus the un-set block at the end) needs to get pulled out into a reusable function and used in those expando functions, too.
</comment_new>

<comment_new>
<comment_id>3406635684</comment_id>
<author>@weswigham</author>
(And `transformCommonJSExport` likely via `transformExpressionStatement`)
</comment_new>

<comment_old>
<author>@copilot</author>
Addressed in d67f7a4d. I removed the global JS-file diagnostic-context assignment and added the missing lazy context setup directly before the CommonJS named-export resolver call (`ensureType(...)`), then restored the previous callback after the call.
</comment_old>

</comment_thread>

</pr_reviews>

</comments>

The last **4** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement><rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. Treat <comment_old> comments as context only and do not make changes solely to satisfy them.
* Only act on <comment_new> comments that explicitly mention you (@copilot). To understand the context of comments that

explicitly mention you, you may use other comments that do not explicitly mention you. Do not act on comments that don't explicitly mention you or reply to them, even if they are marked as <comment_new>.
* If the PR author has pushed back on or rejected a suggestion in a thread, do not act on that thread.
* For each comment you decide to act on, determine:
    - If it is a request for changes, a question on the code, or a suggestion.
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

<system_reminder>

</system_reminder>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.